

Sesión 9: Árboles AVL y Rotaciones Simples

Optimización del Factor de Equilibrio, Modelado Matemático de Alturas e Implementación de Reestructuraciones RLL y RRD en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

Agenda del Día (1/2)

- ⚖ El Problema de la Degradación Lineal en los ABB
- 🌲 Introducción a los Árboles Adelson-Velskii e Landis (AVL)
- ↕ Cálculo de Alturas en Estructuras de Datos Jerárquicas
- 📊 Definición y Ecuación del Factor de Equilibrio (FE)



Balanceo Automático

Descubriremos cómo mantener la complejidad logarítmica $O(\log n)$ real mediante transformaciones dinámicas de punteros en memoria.

Agenda del Día (2/2)



Rotación RLL

Mecánica y código para corregir el desbalanceo izquierdo-izquierdo en el árbol.



Rotación RRD

Mecánica y código para corregir el desbalanceo derecho-derecho en la estructura.



Responsabilidad

Valor de la unidad: Diseñar código predecible y robusto ante flujos masivos de datos.

La Necesidad del Auto-Balanceo

Por qué un ABB tradicional puede fallar estrepitosamente en entornos de alto rendimiento

El Peor Caso de un ABB: Degradación Lineal

Un Árbol Binario de Búsqueda promete búsquedas eficientes. Sin embargo, si insertamos datos ya ordenados (ej. 10, 20, 30, 40), el árbol se inclina por completo hacia un lado.

Pérdida de Rendimiento: El árbol se convierte conceptualmente en una **lista enlazada simple**. La complejidad de búsqueda se degrada de un óptimo $O(\log n)$ a un ineficiente e inaceptable $O(n)$.



¿Qué es un Árbol AVL?

Inventado en 1962 por los matemáticos soviéticos ****Gueorgui Adelson-Velskii**** e ****Evgueni Landis****, representa el primer árbol binario de búsqueda auto-balanceable de la historia de la computación.

Condición Estructural AVL:

Para cualquier nodo del árbol, la diferencia entre las alturas de su subárbol izquierdo y su subárbol derecho no puede ser mayor a 1 ni menor a -1.

Métricas Estructurales: Altura y FE

Los modelos matemáticos internos que permiten detectar desviaciones en el árbol

La Propiedad Altura de un Nodo

La altura de un nodo es la longitud del camino más largo desde ese nodo hasta una hoja terminal.

Para procesarla de manera eficiente en código recursivo, adoptamos las siguientes convenciones matemáticas estándar:

- Un nodo **hoja** tiene una altura de **0**.
- Un puntero o subárbol vacío (**null**) tiene una altura de **-1**.
- La altura de cualquier nodo intermedio se calcula con la fórmula:

$$\text{Altura}(\text{nodo}) = 1 + \max(\text{Altura}(\text{nodo.Izquierdo}), \text{Altura}(\text{nodo.Derecho}))$$

El Factor de Equilibrio (FE)

Es el indicador numérico clave que determina si un nodo está balanceado o si requiere una rotación inmediata.

$$FE = \text{Altura}(\text{Subárbol Derecho}) - \text{Altura}(\text{Subárbol Izquierdo})$$

Un nodo se considera perfectamente **válido y en equilibrio** si su $FE \in \{-1, 0, 1\}$. Si el FE llega a ser **-2** o **2**, el nodo está desbalanceado y debe reestructurarse en ese mismo instante.

Modelado del Nodo AVL en C#

A diferencia del ABB tradicional, el nodo AVL requiere almacenar explícitamente su **altura** para evitar calcularla desde cero en cada operación:

```
public class NodoAVL {  
    public int Dato { get; set; }  
    public NodoAVL Izquierdo { get; set; }  
    public NodoAVL Derecho { get; set; }  
    public int Altura { get; set; } // Propiedad de control esencial  
  
    public NodoAVL(int dato) {  
        Dato = dato;  
        Altura = 0; // Todo nodo nuevo nace como una hoja (altura 0)  
    }  
}
```


Métodos Auxiliares de Control Estructural

Rutinas de protección contra referencias nulas (null) al consultar datos:

```
private int ObtenerAltura(NodoAVL nodo) {  
    return nodo == null ? -1 : nodo.Altura;  
}  
  
private int ObtenerFactorEquilibrio(NodoAVL nodo) {  
    if (nodo == null) return 0;  
    return ObtenerAltura(nodo.Derecho) - ObtenerAltura(nodo.Izquierdo);  
}
```

Rotación Simple a la Izquierda (RLL)

Mecánica de corrección para el desbalanceo crítico en la rama derecha-derecha

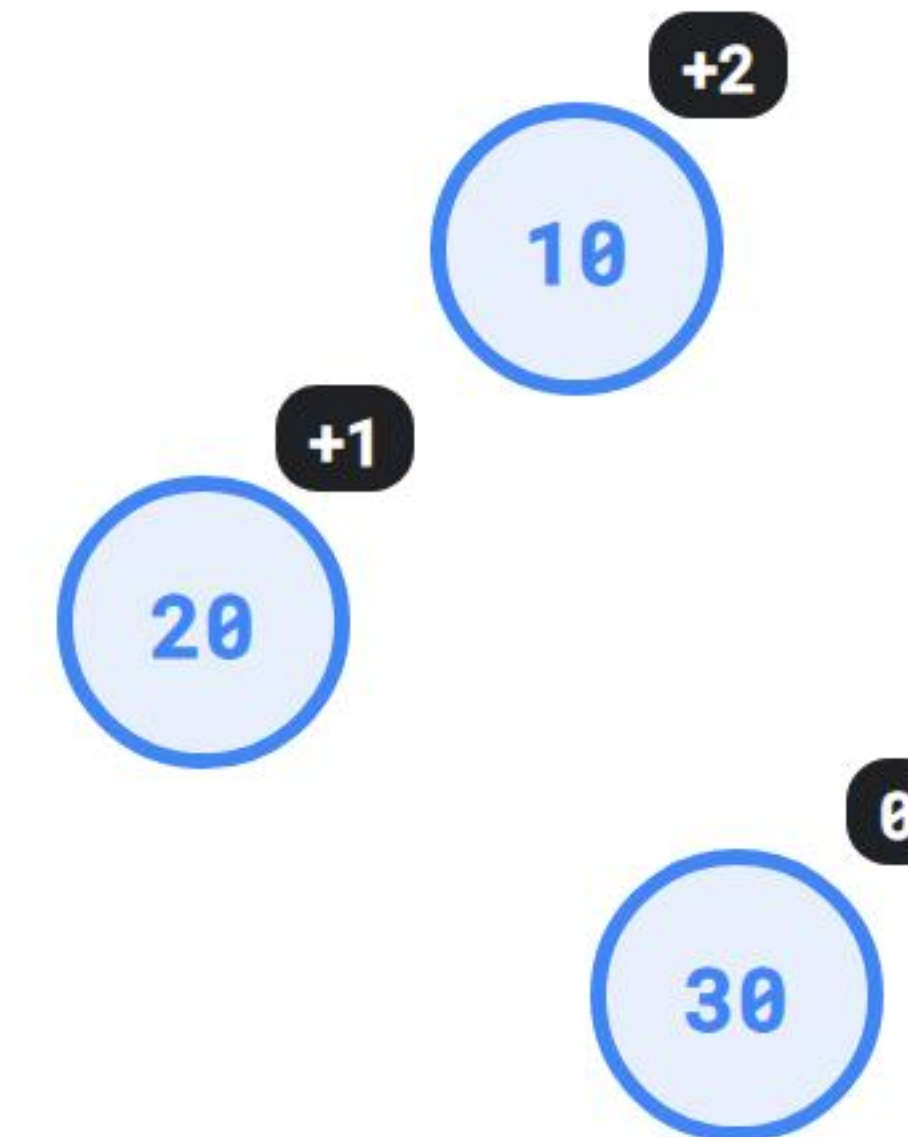
Rotación Simple a la Izquierda (RLL)

Se aplica cuando un nodo tiene un ****Factor de Equilibrio de +2**** y su hijo derecho tiene un ****Factor de Equilibrio de +1 o 0**** (Caso de desbalanceo Derecho-Derecho).

Mecánica del Intercambio:

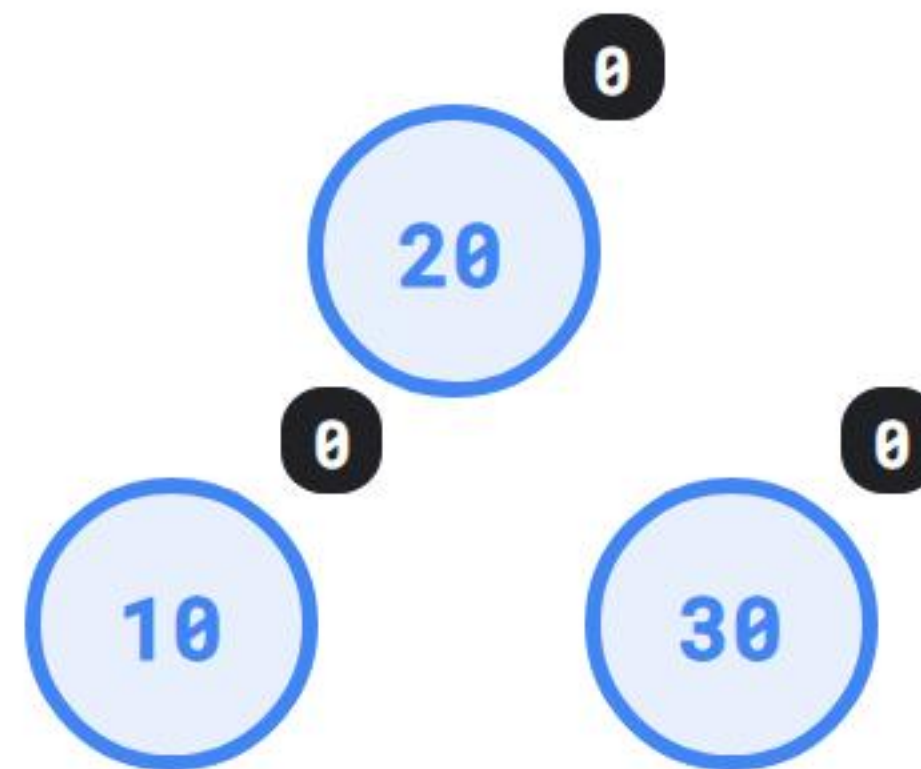
El hijo derecho se convierte en la nueva raíz de ese subárbol. El padre original pasa a ser el nuevo hijo izquierdo del que antes era su hijo.

Si el hijo derecho ya tenía un subárbol izquierdo, este se transfiere y se conecta como el subárbol derecho del padre original.



Resultado de aplicar la Rotación RLL

Observemos cómo la estructura recupera una simetría perfecta tras rotar los punteros hacia la izquierda:



✓ ¡Estructura balanceada! Complejidad de búsqueda restaurada a $O(\log n)$.

Código C#: Implementación de la Rotación RLL

```
private NodoAVL RotarIzquierda(NodoAVL nodoPadre) {
    NodoAVL nuevaRaiz = nodoPadre.Derecho;
    NodoAVL subArbolIzquierdo = nuevaRaiz.Izquierdo;

    // Reestructuración física de los punteros en la memoria heap
    nuevaRaiz.Izquierdo = nodoPadre;
    nodoPadre.Derecho = subArbolIzquierdo;

    // Recálculo estricto de alturas empezando por el nodo que bajó
    nodoPadre.Altura = Math.Max(ObtenerAltura(nodoPadre.Izquierdo), ObtenerAltura(nodoPadre.Derecho)) + 1;
    nuevaRaiz.Altura = Math.Max(ObtenerAltura(nuevaRaiz.Izquierdo), ObtenerAltura(nuevaRaiz.Derecho)) + 1;

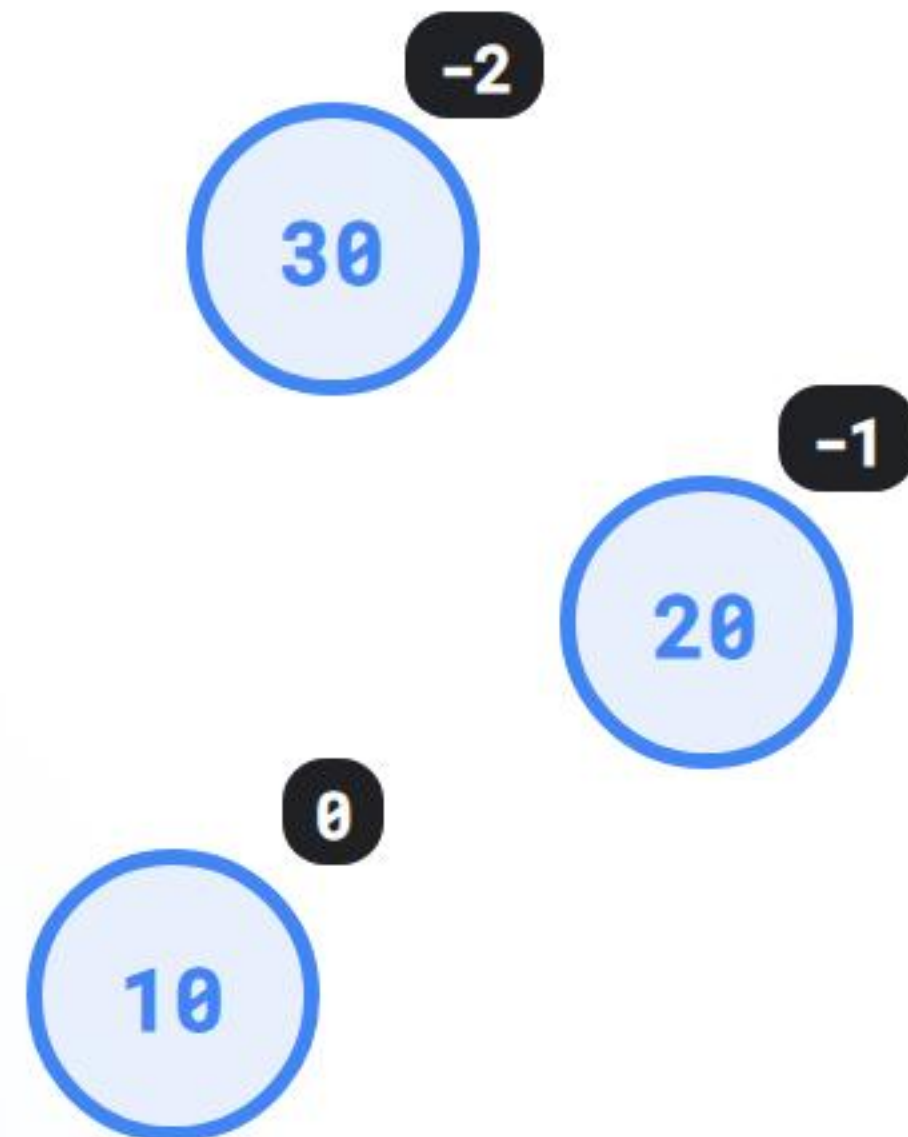
    return nuevaRaiz; // Devuelve la referencia de la nueva raíz local
}
```

Rotación Simple a la Derecha (RRD)

Mecánica de corrección para el desbalanceo crítico en la rama izquierda-izquierda

Rotación Simple a la Derecha (RRD)

Se aplica cuando un nodo tiene un **Factor de Equilibrio de -2** y su hijo izquierdo tiene un **Factor de Equilibrio de -1 o 0** (Caso de desbalanceo Izquierda-Izquierda).



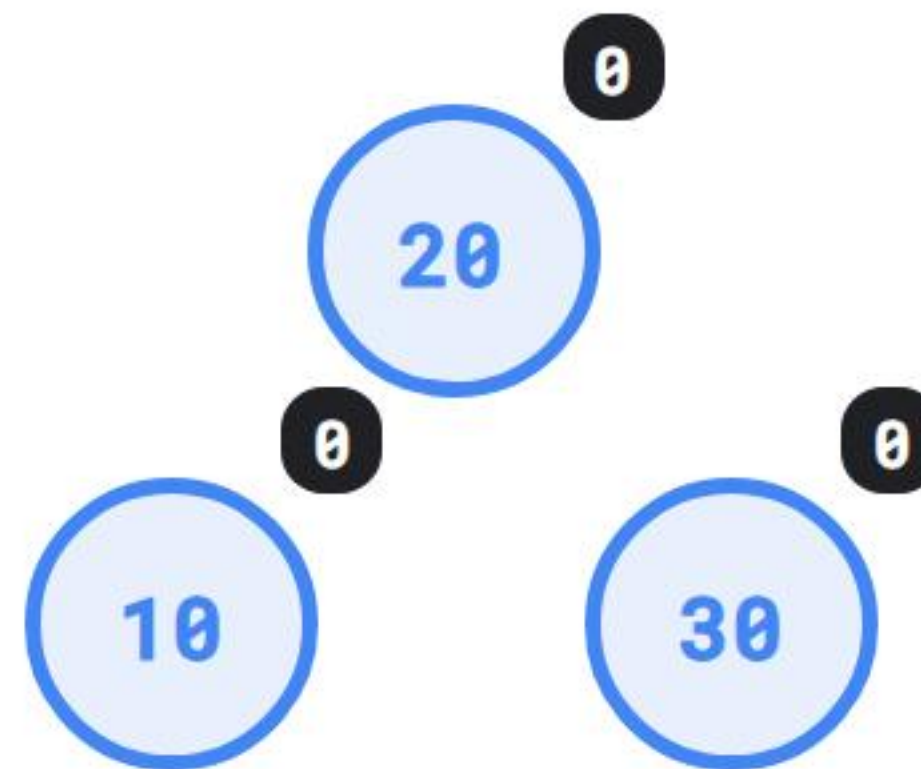
Mecánica del Intercambio:

El hijo izquierdo pasa a ser la nueva raíz de ese subárbol. El padre original desciende y se convierte en el nuevo hijo derecho de su propio ex-hijo.

Si el hijo izquierdo tenía un subárbol derecho original, este se transfiere y se conecta como el subárbol izquierdo del padre que acaba de descender.

Resultado de aplicar la Rotación RRD

Al rotar los punteros físicamente hacia la derecha, el sistema recupera su balance simétrico instantáneamente:



✓ ¡Propiedad AVL satisfecha! Desbalanceo resuelto con éxito.

Código C#: Implementación de la Rotación RRD

```
private NodoAVL RotarDerecha(NodoAVL nodoPadre) {  
    NodoAVL nuevaRaiz = nodoPadre.Izquierdo;  
    NodoAVL subArbolDerecho = nuevaRaiz.Derecho;  
  
    // Reestructuración física de los punteros en la memoria heap  
    nuevaRaiz.Derecho = nodoPadre;  
    nodoPadre.Izquierdo = subArbolDerecho;  
  
    // Recálculo estricto de alturas empezando por el nodo que bajó  
    nodoPadre.Altura = Math.Max(ObtenerAltura(nodoPadre.Izquierdo), ObtenerAltura(nodoPadre.Derecho)) + 1;  
    nuevaRaiz.Altura = Math.Max(ObtenerAltura(nuevaRaiz.Izquierdo), ObtenerAltura(nuevaRaiz.Derecho)) + 1;  
  
    return nuevaRaiz; // Devuelve la referencia de la nueva raíz local  
}
```

Inserción Balanceada en Árboles AVL

Integración de la lógica de balanceo dinámico en la cascada de retorno recursivo

Código: Entrada y Flujo de Inserción AVL

Al igual que en las estructuras anteriores, encapsulamos la raíz protegiendo el estado interno del objeto:

```
public void Insertar(int valor) {  
    raiz = InsertarRecursivo(raiz, valor);  
}
```

La magia del árbol AVL ocurre durante el ****retorno de la recursividad****, donde los nodos revisan y recalculan sus factores de equilibrio de abajo hacia arriba.

Código: Inserción Básica Tipo ABB (1/3)

```
private NodoAVL InsertarRecursoivo(NodoAVL actual, int valor) {
    if (actual == null) return new NodoAVL(valor);

    // 1. Descenso recursivo tradicional
    if (valor < actual.Dato) {
        actual.Izquierdo = InsertarRecursoivo(actual.Izquierdo, valor);
    } else if (valor > actual.Dato) {
        actual.Derecho = InsertarRecursoivo(actual.Derecho, valor);
    } else {
        return actual; // No se admiten llaves duplicadas en este modelo
    }

    // 2. Actualizar altura del nodo padre local
    actual.Altura = 1 + Math.Max(ObtenerAltura(actual.Izquierdo), ObtenerAltura(actual.Derecho));
    // (Continúa en la siguiente diapositiva...)
```


Código: Evaluación y Rotación Dinámica (2/3)

```
// 3. Evaluar el Factor de Equilibrio para detectar desbalanceos
int fe = ObtenerFactorEquilibrio(actual);

// CASO DE ROTACIÓN SIMPLE A LA DERECHA (RRD)
if (fe < -1 && valor < actual.Izquierdo.Dato) {
    return RotarDerecha(actual);
}

// CASO DE ROTACIÓN SIMPLE A LA IZQUIERDA (RLL)
if (fe > 1 && valor > actual.Derecho.Dato) {
    return RotarIzquierda(actual);
}

// Nota: Los desbalanceos cruzados (izq-der y der-izq) corresponden
// a rotaciones dobles y se implementarán en la Sesión 10.
return actual;
}
```

Simulación Práctica del Algoritmo

Verificación visual y trazabilidad del flujo de datos en la consola de comandos de .NET

Código: Ejecución del Flujo de Pruebas

```
public static void Main(string[] args) {  
    ArbolAVL arbol = new ArbolAVL();  
  
    // Insertamos una secuencia ordenada que forzaría la degradación en un ABB  
    Console.WriteLine("--- Insertando 10, 20, 30 de forma secuencial ---");  
    arbol.Insertar(10);  
    arbol.Insertar(20); // Árbol inclinado a la derecha  
    arbol.Insertar(30); // ¡Aquí se dispara automáticamente el método RotarIzquierda!  
  
    Console.WriteLine("¡Estructura optimizada y balanceada con éxito!");  
}
```

Buenas Prácticas e Ingeniería de Software

Estrategias avanzadas para garantizar el mantenimiento de algoritmos recursivos complejos

Error Común: Olvidar Recalcular la Altura

Un error crítico muy frecuente en los exámenes prácticos del laboratorio es modificar los punteros en las funciones de rotación sin actualizar los enteros de la propiedad `Altura`.

Consecuencia Operativa: El árbol realiza la primera rotación de forma correcta, pero todas las evaluaciones de los factores de equilibrio subsiguientes utilizarán datos obsoletos, rompiendo la lógica AVL en las siguientes inserciones.

💡 **Regla de Oro:** Primero se recalcula la altura del nodo que bajó en la jerarquía, y de último la altura del nodo que subió (la nueva raíz local).

Valor de la Unidad: Responsabilidad



"No se puede escapar de la responsabilidad del mañana evadiéndola hoy."

— Abraham Lincoln

****Aplicación en Ingeniería:**** Diseñar estructuras auto-balanceables eficientes es un acto de responsabilidad profesional. Garantiza que el software mantenga su velocidad y estabilidad sin importar el volumen de datos que cargue el usuario.

Conclusiones de la Sesión

Garantía $O(\log n)$

Los árboles AVL resuelven el peor caso de los ABB, asegurando tiempos de respuesta logarítmicos estables en todas las operaciones.

Métrica del FE

El factor de equilibrio se calcula restando la altura izquierda de la derecha. Solo valores entre -1, 0 y 1 se consideran válidos.

Rotaciones Simples

Las transformaciones RLL y RRD solucionan de forma directa y eficiente desbalanceos puros en ramas lineales de la estructura.

Próximo Paso: Sesión 10

Rotaciones Dobles en Árboles AVL

En la siguiente sesión abordaremos los desbalances combinados o cruzados mediante el estudio e implementación de las ****Rotaciones Dobles**** (Izquierda-Derecha y Derecha-Izquierda) tras operaciones masivas de datos.

¿Dudas sobre Árboles AVL o Rotaciones Simples?

Es momento de actualizar e integrar sus entornos de desarrollo.



Foros en UEDI



Soporte: 3021894750101@ingenieria.usac.edu.gt